

Structuring Code

Name: Quah Xuanyu

Student ID: 35624914

Date: 25/8/2025

Learning Journey and Evidence

Prior Knowledge and Experience

I do have prior experience in functions and procedures, which also include the usage of parameters, return values, and modular programming. I have also learned C++ before, and have been using functions and procedures for a while already.

Study to Acquire, Refresh, and/or Extend Capability

To study and master this topic I did the following:

25/8/2025 - Studied throwing and catching exceptions

```
25
26 ~ int main()
27 {
28     int check;
29
30     try
31     {
32         // Test calling double positive with a positive value
33         check = double_positive(5);
34         // This should succeed, and the doubled value printed
35         write_line("Check is " + to_string(check));
36
37         // Test calling double positive with a negative value
38         check = double_positive(-1);
39     }
40     catch(string e)
41     {
42         // Output the error when it occurs.
43         write_line(e);
44     }
45     catch(...)
46     {
47         // Use ... to catch any other exception.
48         write_line("Some other exception occurred");
49     }
50
51     // See what happens when an exception occurs outside of a handler.
52     check = double_positive(-5);
53
54     return 0;
55 }
```

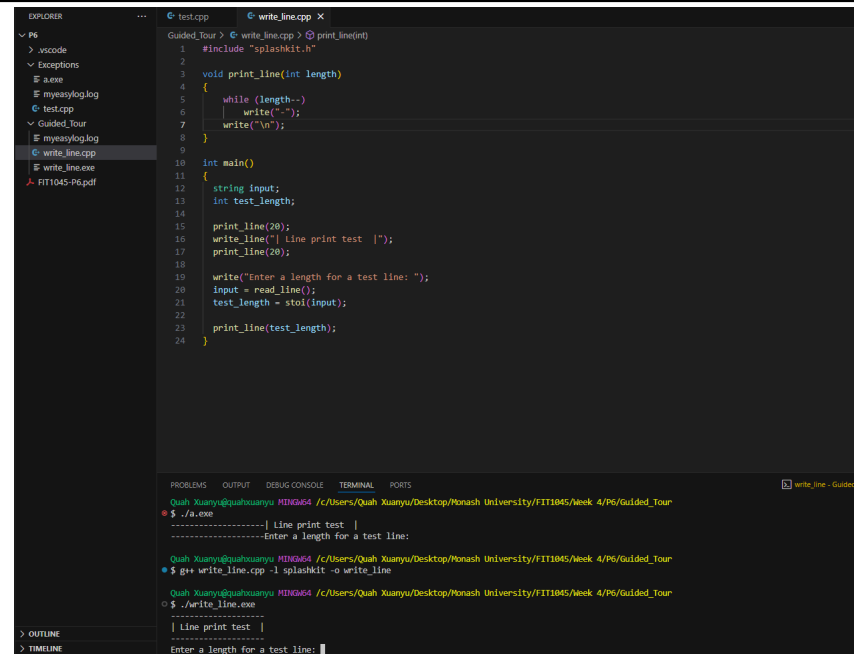
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - exceptions +

```
$ ./a.exe
Check is 10
Value must be positive
terminate called after throwing an instance of 'std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >'
Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/exceptions
$ g++ test.cpp -l splashkit
Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/exceptions
$ ./a.exe
terminate called after throwing an instance of 'std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >'
Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/exceptions
$
```

Throwing exceptions are very useful in error handling and error management. I learned that it is actually much easier to debug and find errors as you can just throw exceptions, and have it be caught and printed out, which makes it very easy to debug.

Guided Tour

25/8/2025 - Wrote the write line procedure



```
EXPLORER
P6
  > .vscode
  > Exceptions
  > myeasylog.log
  > test.cpp
  > Guided_Tour
  > myeasylog.log
  > write_line.cpp
  > write_line.exe
  > FIT1045-P6.pdf

test.cpp
write_line.cpp X
Guided_Tour > > write_line.cpp > > print_line(int)
1 #include "splashkit.h"
2
3 void print_line(int length)
4 {
5     while (length-- > 0)
6     {
7         write("-");
8     }
9     write("\n");
10
11 int main()
12 {
13     string input;
14     int test_length;
15
16     print_line(20);
17     write_line("Line print test ");
18     print_line(20);
19
20     write("Enter a length for a test line: ");
21     input = read_line();
22     test_length = stoi(input);
23
24     print_line(test_length);
25 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Quah Xuanyu@quahxuanyu MINGW64 /C:/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Guided_Tour

• \$./a.exe

-----| Line print test |

-----Enter a length for a test line:

Quah Xuanyu@quahxuanyu MINGW64 /C:/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Guided_Tour

• \$ g++ write_line.cpp -l splashkit -o write_line

Quah Xuanyu@quahxuanyu MINGW64 /C:/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Guided_Tour

○ \$./write_line.exe

-----| Line print test |

-----Enter a length for a test line: |

Here, I learned to create a procedure and take an argument for the length, and print out the length of the repeated character according to the length. This will result in a printed line.

25/8/2025 - Did print repeated text

```

test.cpp x write_line.cpp x
Guided_Tour > write_line.cpp > print_repeated(string, int, bool)
1 #include "splashkit.h"
2
3 void print_repeated(string text, int times, bool newline)
4 {
5     while (times-- > 0)
6     {
7         write(text);
8         if (newline)
9             write("\n");
10    }
11
12 void print_line(int length)
13 {
14     while (length-- > 0)
15     {
16         write("-");
17     }
18     write("\n");
19 }
20
21 int main()
22 {
23     string input;
24     int test_length;
25
26     print_line(20);
27     write_line("| Line print test |");
28     print_line(20);
29
30     print_repeated("---++---", 5, true);
31     print_repeated("Hello World\n", 5, false);
32     print_repeated("---++---", 5, true);
33
34     write("Enter a length for a test line: ");
35     input = read_line();
36     test_length = stoi(input);
37 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Guided_Tour
$ ./write_line.exe
-----
| Line print test |
-----
---++---
Hello World
Hello World
Hello World
Hello World
Hello World
---++---
Enter a length for a test line:

```

Similar to the one above, but instead this prints the same word repeatedly, and it also takes in argument a second argument, which is a boolean variable that tells whether or not to print a new line at the end.

25/8/2025 - Did functions practice

```

test.cpp x write_line.cpp x functions.cpp x
Guided_Tour > functions.cpp > read_integer(string)
1 #include "splashkit.h"
2
3 using std::to_string;
4 using std::stoi;
5
6 string read_string(string prompt)
7 {
8     write(prompt);
9     return read_line();
10 }
11
12 int read_integer(string prompt)
13 {
14     string input = read_string(prompt);
15     while (!is_integer(input))
16     {
17         write_line("Please enter a whole number");
18         input = read_string(prompt);
19     }
20     return stoi(input);
21 }
22
23 // Add read integer code here
24
25 int main()
26 {
27     string name;
28     int age;
29
30     // Read in a string and store it in name
31     name = read_string("enter your name: ");
32
33     // Output the name
34     write_line("Hello " + name);
35 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Guided_Tour
$ g++ functions.cpp -l splashkit -o function
Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Guided_Tour
$ ./function.exe
enter your name: john
Hello john
what is your age: kok
Please enter a whole number
what is your age: 1231
You entered 1231
Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Guided_Tour
$

```

Practiced with some more functions

Guess the Number game

25/8/2025 - Started making Guess the number game

```

1  #include "splashkit.h"
2
3  string read_string(string prompt)
4  {
5      write(prompt);
6      return read_line();
7  }
8
9  int read_integer(string prompt)
10 {
11     string input = read_string(prompt);
12     while (!is_integer(input))
13     {
14         write_line("Please enter a whole number");
15         input = read_string(prompt);
16     }
17     return stoi(input);
18 }
19
20
21 /**
22  * Play one game of 7 rounds
23  */
24 bool guess()
25 {
26
27 }
28
29
30 /**
31  * Keep repeating until stop playing
32  */
33 int main(void)
34 {
35
36 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

$ rm Guess_Number

Quah Xuanyu@quahxuanyu MINGW64 /c:/Users/Quah Xuanyu/Desktop/Monash University/FT11045/Week 4/P6/Guided_Tour
$ touch Guess_Number.cpp

Quah Xuanyu@quahxuanyu MINGW64 /c:/Users/Quah Xuanyu/Desktop/Monash University/FT11045/Week 4/P6/Guided_Tour

```

First, I listed out the functions needed first, before actually implementing them. This includes one `guess()` function, that return true/false.

25/8/2025 - Created guess_num() function

```
/**
 * Prompt guess and checks
 */
bool guess_num(int input, int target)
{
    if (input < target)
        write_line("The number is more than " + to_string(input));
    else if (input > target)
        write_line("The number is less than " + to_string(input));
    else
    {
        write_line("Well done! The number is " + to_string(input));
        return true;
    }
    return false;
}
```

Given the target and the input, compare both of them to see which is higher and which is lower, or equal, and print its respective message. Then return either true/false depending on whether it is correct or not.

25/8/2025 - Created play_game() function

```
/**
 * Play one game of 7 rounds
 */
void play_game()
{
    int i = 0;
    int target = rnd(0, 100);
    string guess;
    write_line("I am thinking of a number between 1 and 100\n");
    while (i++ < 7)
    {
        guess = read_string("Guess " + to_string(i) + ": ");
        while (!is_integer(guess))
        {
            write_line("Invalid input, try again");
            guess = read_string("Guess " + to_string(i));
        }
        if (guess_num(stoi(guess), target))
            return ;
    }
    write_line("You ran out of guesses... the number was " + to_string(target));
}
```

The play game function will create the random number, and allow the user to guess up to 7 rounds. This function also calls the guess number function and the

read string function. If the player guesses the number correctly, it will return and stop the function, else the while loop will continue looping until 7 guesses are up.

25/8/2025 - Errors: Used Or instead of And

```
/**
 * Keep repeating until stop playing
 */
int main(void)
{
    string input;
    do
    {
        play_game();
        input = read_string("Do you want to play again [Y/n]? ");
    } while (input != "N" | input != "n");
}
```

The main function just has a do while loop to check whether the player still wants to continue to play after a game is over. It will call the play game function.

25/8/2025 - Error: Mixed up less than and more than

```
/**
 * Prompt guess and checks
 */
bool guess_num(int input, int target)
{
    if (input < target)
        write_line("The number is less than " + to_string(input));
    else if (input > target)
        write_line("The number is more than " + to_string(input));
    else
    {
        write_line("Well done! The number is " + to_string(input));
        return true;
    }
    return false;
}
```

After compiling and running the code, I realised that the numbers didn't seem right. Upon debugging, I realised that I flipped the ">" and "<" for the if statements, which resulted in when the number is actually bigger, it says it is lower. Flipping it around solved this issue.

25/8/2025 - Used break, instead of return

```
Well done! The number is 47
You ran out of guesses... the number was 47
Do you want to play again [Y/n]? I am thinking of a number between 1 and 100
```

```
    }
    if (guess_num(stoi(guess), target))
        break ;
}
write_line("You ran out of guesses... the number was " + to_string(target));
```

```
    }
    if (guess_num(stoi(guess), target))
        return ;
}
write_line("You ran out of guesses... the number was " + to_string(target));
```

Used break instead of return, causing it to print correct, yet wrong also.

To mitigate this, I used an early return to stop the procedure, and end it early to prevent printing it unnecessarily.

25/8/2025 - Converted "Change Calculator"

```
1 #include "splashkit.h"
2
3 using std::stoi;
4 using std::to_string;
5
6
7 string read_string(string prompt)
8 {
9     write(prompt);
10    return read_line();
11 }
12
13 int read_integer(string prompt)
14 {
15     string input = read_string(prompt);
16     while (!is_integer(input))
17     {
18         write_line("Please enter a whole number");
19         input = read_string(prompt);
20     }
21     return stoi(input);
22 }
23
24 int main()
25 {
26     string line;
27     string coinText;
28     int itemCost, amountPaid, change, toGive, coinValue;
29     const int TWO_DOLLARS = 200;
30     const int ONE_DOLLARS = 100;
31     const int FIFTY_CENTS = 50;
32     const int TWENTY_CENTS = 20;
33     const int TEN_CENTS = 10;
34     const int FIVE_CENTS = 5;
35     const int ONE_CENTS = 1;
36     const int NO_TYPES_COINS = 7;
37
38     do
39     {
40         itemCost = read_integer("Cost of item (in cents): ");
41         amountPaid = read_integer("Amount paid (in cents): ");
42         if (amountPaid < itemCost)
43             write_line("Insufficient payment");
44         else
45         {
46             change = amountPaid - itemCost;
47             write("Change: ");
48             for (int i = 0; i < NO_TYPES_COINS; i++)
```

```
Amount paid (in cents): 123112
Change: 609 x $2, 0 x $1, 1 x 50c, 1 x 20c, 1 x 10c, 0 x 5c, 0 x 1c
Run again? Y/n: Cost of item (in cents):

Amount paid (in cents): 123112
Change: 609 x $2, 0 x $1, 1 x 50c, 1 x 20c, 1 x 10c, 0 x 5c, 0 x 1c
Run again? Y/n: Cost of item (in cents):

Change: 609 x $2, 0 x $1, 1 x 50c, 1 x 20c, 1 x 10c, 0 x 5c, 0 x 1c
Run again? Y/n: Cost of item (in cents):

Run again? Y/n: Cost of item (in cents):

Quah Xuanyu@quahxuanyu MINGW64 /C:/Users/Quah Xuanyu/Desktop/Monash Unive
rsity/FT11045/Week 4/P6/Change_Calculator
$
```

Here, I applied the `read_string()` and `read_integer()` functions I created to the Change Calculator program from the previous exercise. This allowed the code to be less cluttered and more readable, as there is less repeated segments of code.

25/8/2025 - Created `give_change()` function


```
void give_change(int change)
{
    int toGive, coinValue;
    string coinText;
    const int TWO_DOLLARS = 200;
    const int ONE_DOLLARS = 100;
    const int FIFTY_CENTS = 50;
    const int TWENTY_CENTS = 20;
    const int TEN_CENTS = 10;
    const int FIVE_CENTS = 5;
    const int ONE_CENTS = 1;
    const int NO_TYPES_COINS = 7;

    write("Change: ");
    for (int i = 0; i < NO_TYPES_COINS; i++)
    {
        switch (i)
        {
            case 0:
                coinValue = TWO_DOLLARS;
                coinText = "$2, ";
                break;
            case 1:
                coinValue = ONE_DOLLARS;
                coinText = "$1, ";
                break;
            case 2:
                coinValue = FIFTY_CENTS;
                coinText = "50c, ";
                break;
            case 3:
                coinValue = TWENTY_CENTS;
                coinText = "20c, ";
                break;
            case 4:
                coinValue = TEN_CENTS;
                coinText = "10c, ";
                break;
            case 5:
                coinValue = FIVE_CENTS;
                coinText = "5c, ";
                break;
            case 6:
                coinValue = ONE_CENTS;
                coinText = "1c, ";
                break;
        }
    }
}
```

I also split the printing the change section to a separate procedure, also declutters the main function and allows a simple call of the give_change() procedure, and gives it the change needed.

```
#include "splashkit.h"

using std::to_string;

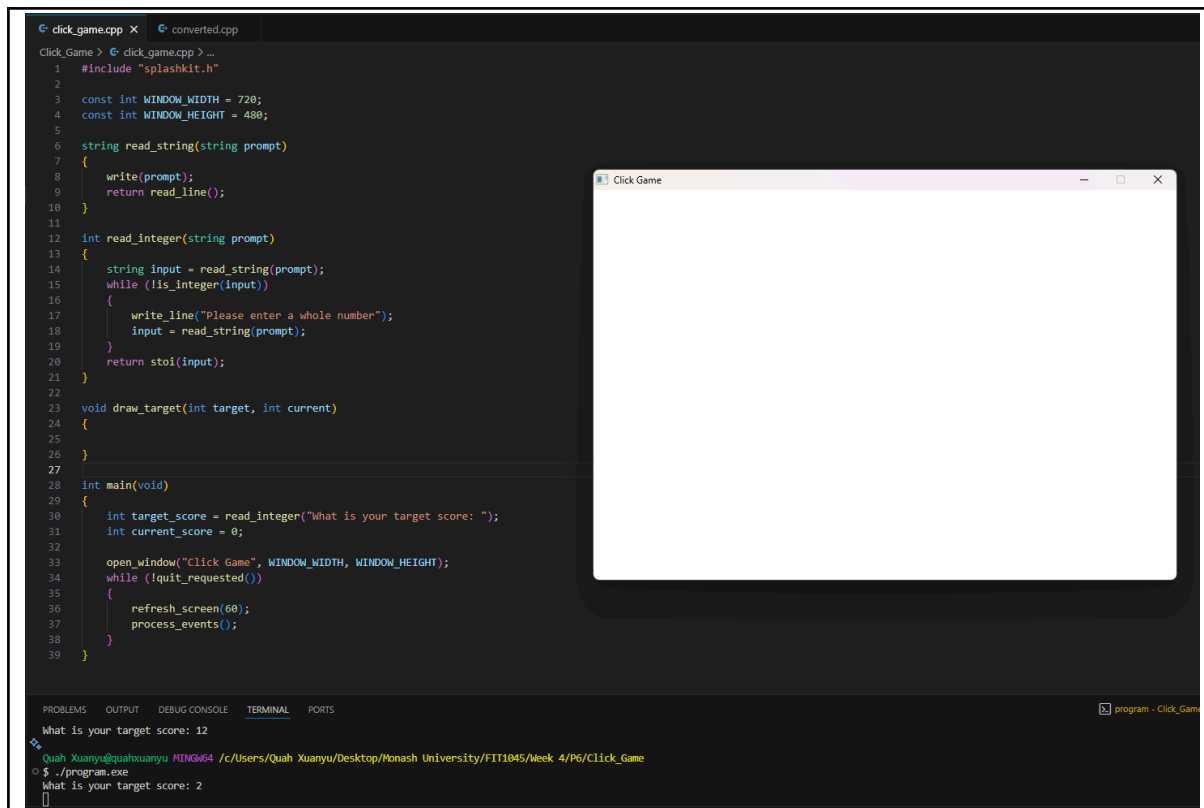
const int PLAYER_RADIUS = 50;
const int WINDOW_WIDTH = 720;
const int WINDOW_HEIGHT = 480;
const int FLY_APPEAR = 1500; // Appear for 1.5 seconds
int PLAYER_SPEED = 5;

void draw_game(int player_x, int player_y, bool flyAppear, int flyX, int flyY, int score)
{
    clear_screen(color_white());
    if (flyAppear)
    {
        fill_circle(color_yellow(), flyX, flyY, 20);
    }
    fill_circle(color_blue(), player_x, player_y, PLAYER_RADIUS);
    draw_text("Flies Caught: " + to_string(score), color_black(), 10, 20);
    refresh_screen();
    refresh_screen(60);
    process_events();
}
```

For this, I just took out the filling up the screen of the splashkit window section into a separate function. Once again, this allows the main function to look less cluttered, and more readable.

Test Your Knowledge

25/8/2025 - Created the base template



```

1 #include "splashkit.h"
2
3 const int WINDOW_WIDTH = 720;
4 const int WINDOW_HEIGHT = 480;
5
6 string read_string(string prompt)
7 {
8     write(prompt);
9     return read_line();
10 }
11
12 int read_integer(string prompt)
13 {
14     string input = read_string(prompt);
15     while (!is_integer(input))
16     {
17         write_line("Please enter a whole number");
18         input = read_string(prompt);
19     }
20     return stoi(input);
21 }
22
23 void draw_target(int target, int current)
24 {
25 }
26
27
28 int main(void)
29 {
30     int target_score = read_integer("What is your target score: ");
31     int current_score = 0;
32
33     open_window("Click Game", WINDOW_WIDTH, WINDOW_HEIGHT);
34     while (!quit_requested())
35     {
36         refresh_screen(60);
37         process_events();
38     }
39 }

```

Click Game > click_game.cpp > ...

What is your target score: 12

Quah Xuanyu@quahxuanyu MINGW64 /c/Users/Quah Xuanyu/Desktop/Monash University/FIT1045/Week 4/P6/Click_Game

o \$./program.exe

What is your target score: 2

program - Click_Game

First, I created the `read_string()` and `read_integer()` function, as I know I will be needing it later. Then I know I will split the drawing target portion into a separate function `draw_target()`. Inside the main function, I just created a simple screen for now.

25/8/2025 - Added drawing for ball and bar

```

void draw_healthbar(int target, int current, double enemyX, double enemyY)
{
    fill_rectangle(color_brown(), enemyX - 45, enemyY - 45, BAR_WIDTH, BAR_HEIGHT);
    fill_rectangle(color_green(), enemyX - (45 - (INNER_BAR_OFFSET / 2)), enemyY - (45 - (INNER_BAR_OFFSET / 2)), BAR_WIDTH - INNER_BAR_OFFSET, BAR_HEIGHT - INNER_BAR_OFFSET);
}

void draw_target(int target, int current)
{
    double enemyX = rnd(BOUNDARY, WINDOW_WIDTH - BOUNDARY);
    double enemyY = rnd(BOUNDARY, WINDOW_HEIGHT - BOUNDARY);
    draw_healthbar(target, current, enemyX, enemyY);
    fill_circle(color_black(), enemyX, enemyY, 20);
}

```



I created another procedure, called `draw_healthbar()` which draws the health bar of the target base on its position. The draw target will first generate a random position within the window size and then draw a black circle. Then, it will call the `draw_healthbar()` procedure to draw it base on the position of the circle.

25/8/2025 - Added clicking mechanism

```

int main(void)
{
    int target_score = read_integer("What is your target score: ");
    int current_score = 0;
    double enemyX = rnd(BOUNDARY, WINDOW_WIDTH - BOUNDARY);
    double enemyY = rnd(BOUNDARY, WINDOW_HEIGHT - BOUNDARY);

    open_window("Click Game", WINDOW_WIDTH, WINDOW_HEIGHT);
    draw_target(target_score, current_score, enemyX, enemyY);
    while (!quit_requested())
    {
        if (point_in_circle(mouse_position().x, mouse_position().y, enemyX, enemyY, 20) && mouse_clicked(LEFT_BUTTON))
        {
            current_score++;
            enemyX = rnd(BOUNDARY, WINDOW_WIDTH - BOUNDARY);
            enemyY = rnd(BOUNDARY, WINDOW_HEIGHT - BOUNDARY);
            clear_screen();
            draw_target(target_score, current_score, enemyX, enemyY);
        }
        refresh_screen(60);
        process_events();
    }
}

```

- I decided to move the position of the enemy into the main function, as I will need it
- I added a new if statement in the while loop to check for clicking of the mouse, and whether it is inside the enemy position.
- if It is, it will increase the score, and change to position of the enemy to another random one
- it will clear the screen, and draw the target again.

25/8/2025 - Decrease healthbar length based on score

```

void draw_target(int target, int current, double enemyX, double enemyY)
{
    double innerBarWidth = (BAR_WIDTH - INNER_BAR_OFFSET) * (1 - ((double) current / target));
    double innerBarHeight = BAR_HEIGHT - INNER_BAR_OFFSET;

    //Draw Health Bar
    fill_rectangle(color_brown(), enemyX - 45, enemyY - 45, BAR_WIDTH, BAR_HEIGHT);
    fill_rectangle(color_green(), enemyX - (45 - (INNER_BAR_OFFSET / 2)), enemyY - (45 - (INNER_BAR_OFFSET / 2)), innerBarWidth, innerBarHeight);

    //Draw Enemy
    fill_circle(color_black(), enemyX, enemyY, 20);
}

```



- Combined draw target and draw healthbar, cus its redundant
- Calculate inner bar length base on target hit and current score
 - This is done by multiplying the bar with the proportion of the current score and the target score, which will determine how much health the target has left.
- Ends the game when score reached

25/8/2025 - Added score counter on the side

```
✓ void draw_score(int current_score, int target_score)
{
    draw_text("Score: " + to_string(current_score), color_black(), 10, 10);
    draw_text("Target: " + to_string(target_score), color_black(), 10, 30);
}
```

Score: 3
Target: 10



At the end, I added another function to draw the score, which will also be added inside the while loop if statement and be updated whenever the enemy is clicked.

Thus, the clicking game is complete!

Brief Summary of Concepts

Concept	Key Idea / Concept
Procedure	A procedure is a block of code or series of instructions that can be called and executed from other parts of the code. The main usage of procedures is to reduce repeated segments of code that are used several times in the program. This improves readability and ease for debugging.
Function	A function is similar to a procedure, where it will also contain a series of instructions that will be executed when called, but it will also return a value when the function ends.
Return	Return is a keyword used to end a procedure and function, and it may also return a value. - For procedures, the return must not have a value, which can allow the procedure to end early - For functions, it must have a return and it must return a value
Local Variable	Local variables are variables that only exist within the scope defined. For instance, the local variable in functionA is not accessible in functionB, as it is out of scope when the program is running in functionB. Local variables are stored on the stack , rather than the heap , as no memory is allocated for it. When the function ends, the local variable is destroyed.
Parameter	Parameters are values that are passed into a function or procedure that is used as a variable in the function or procedure. This is used to provide specifications for the instructions that will be run, which will affect the output of the procedure or function. For instance, the X and Y positions of the fill_circle() procedure is given as a parameter.
Global Variable	Global Variables are variables declared outside of any functions or procedures, and can be accessed by any functions or procedures throughout the file. However, it should be avoided as it can be difficult to trace the origin or errors, as the variables can be accessed by any functions. Global variables should only be used to declare constants .

Reflection

What is the most important thing you learned from this and why?

The most important thing I learned in this chapter was learning more about the splashkit library and how to make games out of it. I also learned to throw and catch exceptions, which I previously did not have much experience using.

How do functions and procedures help you approach the development of larger programs?

It allows me to think in terms of smaller parts, where I break the program up into different smaller functionalities, and make different procedures and functions based on that. Rather than storing everything in one large function, this allows the code to be easier to understand and easier to work with.

How do parameters make your functions and procedures more useful? Give examples from your work.

Parameters are crucial when making functions and procedures, as it alters the behavior and give out specific actions based on the parameters, making one function or procedure more versatile and useful. Without parameters, the function would only be able to do one set of instructions, with no variation to it every time you call it. For example, I made the `draw_target()` function in the clicking game, where it receives the parameters of the current score, target score, X, and Y positions. The procedure will then draw the target base on those inputs, and by simply altering the parameters, we can change the output of the procedure, without the need to create an entirely new procedure.

Why are global variables considered poor practice, when global constants are considered good practice?

Global variables can be altered by any function or procedure, which can change its value. It should be avoided as it can be difficult to trace the origin or errors. Global variables should only be used to declare **constants**, as they are used throughout the program regularly, but they are not changed so it is perfectly acceptable.

References

Generative AI acknowledgement

I did not use any generative AI for this project.